P0221R1
revises: P0221R0
Jens Maurer <Jens.Maurer@gmx.net>
Audience: Core Working Group
2016-03-17

P0221R1: Proposed wording for default comparisons, revision 3

Introduction

This paper presents design rationale and proposed wording to implement default comparisons for class types. It is a revision of N4532 with additional updates from the Evolution Working Group session at the Kona meeting of WG21 and in-depth discussions with interested parties.

This paper assumes that the reader is familar with N4475 "Default comparisons (R2)" by Bjarne Stroustrup. In particular, default comparisons are assumed to be implicit (i.e. require no extra syntax to be available).

P0221R0 amended by a clarification for template specializations was approved by EWG during the Jacksonville (2016-03) meeting of WG21. Blue text in the proposed wording indicates changes compared to P0221R0.

Changes since P0221R0

- fix wording glitches
- clarified the zero-subobject (= empty class) cases
- clarified that the context for a subobject comparison is the top-level class definition
- clarified that both slicing copy and slicing move are prohibited
- two-phase name lookup applies when comparing members whose type was instantiated from a dependent type

Design

The fundamental open design questions are:

- When exactly is a default comparison chosen over a user-declared one?
- Once chosen, what exactly are the semantics of the default comparison? In particular, which comparison operators on the subobjects are looked up where?

Aims

These are general high-level aims and are not all fully satisfied by the design presented further below. The numbering does not indicate a priority, but is intended for easier reference.

1. A class author should have no valid reason to write a comparison function by hand if the default semantics suffice.
2. A default comparison yields the same result for the same arguments, regardless where the default comparison appears (consistency).
3. Do not invalidate existing code.
4. Prevent slicing of objects.
5. Do not impose substantial new burdens on implementers.
6. Make comparisons and copy-assignment similar.

In an ideal world, the comparison operators would be declared alongside the class they apply to, i.e. as members or (preferrably) as non-members in the immediately enclosing namespace of the class. Unconstrained template parameters in comparison operators or declarations in unrelated namespaces are considered a bug in the user's code. That said, in order to avoid breaking existing code, existing semantics of code are preserved by this proposal as much as possible without jeopardizing the other aims. If in doubt, an error is generated instead of silently using a default comparison.

Given aim 3, we're 30+ years late in mandating the ideal world, which is too late. In particular, that means we cannot enforce the important principle that x==y appearing in different corners of the source code has the same semantics everywhere. We can, however, make sure that the default (if used) has the same semantics everywhere.

Rules

The following rules reflect the aims above. Note that they change the meaning of existing (arguably broken) code, as highlighted in the examples below. These rules are given in informal language; for a precise wording of the rules, refer to the "wording" section below.

First, we modify the existing rules for overload resolution of the =, equality, and relational operators as well as copy construction. This is intended to prevent slicing (aim 4, aim 6). We allow for benign slicing when the derived class adds no data members (aim 3), which is needed to continue to support tag dispatch for standard library algorithms.

> R1: For a class B and a class D derived from B, attempting to bind a `const B&` parameter of the copy constructor or of the operator=, equality, or relational functions to a value of type D makes overload resolution fail, unless all other (if any) base classes of D are empty and the definition of D declares no data members and no virtual functions.

Second, we introduce declarations (but not definitions) of the default comparisons for each class that is defined (aim 1). Note that a class template specialization is a class that is defined when the class template is instantiated or explicitly specialized. A friend declaration is only visible to argument-dependent lookup, thus we can only find it for function calls, but not for taking the address.

> R2: For each class that is defined, equality (5.10 expr.eq) and relational (5.9 expr.rel) operator functions according to the pattern
>
> `bool operator` *`op`*`(const C&, const C&);`
>
> are implicitly declared (unless there was a preceding user declaration with a similar signature), but not yet defined. (A member declaration of *op* is transformed into the equivalent non-member form by introducing the implicit object parameter (13.3.1 over.match.funcs) for the check.) The declaration is as-if by a friend declaration immediately before the closing brace of the class definition.

Third, we allow a user declaration of one of those operator functions to hide the implicitly-declared one (aim 3):

> R3: A user-declared comparison function hides the corresponding implicitly-declared one for class C with a similar signature (if any). Hiding means that if both appear in a lookup set, only the user-declared comparison function is considered in overload resolution.

Fourth, the definition of a default comparison (aim 2, aim 6):

> R4: An implicitly-declared comparison function for a class C is defined if it is odr-used (3.2 basic.def.odr). It performs subobject decomposition and then subobject comparisons (see "wording" below). The subobject comparisons are performed in the context of class C as-if immediately before the closing brace of the class definition. If subobject comparison would be ill-formed, the comparison is defined as deleted.

Fifth, using both a default comparison and the corresponding user-declared comparison is a syntax error (aim 2):

> R5: If an expression or the definition of a default comparison uses a default comparison, but would use a user-declared comparison if the former appeared later in the translation unit, the program is ill-formed. No diagnostic is required if the use and the competing declaration are in different translation units.

Sixth, do not apply user-defined conversions for a homogenous comparison:

> R6: In a comparison applied to operands of the same type, no user-declared conversions are considered when converting the operands to the parameter types of the comparison operator function.

Seventh, define "similar signature". *This could be extended to (some kinds of) function templates if desired.*

> Two operator functions have a *similar signature* if they
>
> - have the same name and,
> - given a class type C and the types T1 and T2 of corresponding parameters, each of T1 and T2 is either C or reference to *cv* C

Note that the wording below takes a slightly different approach by reusing the intermediate results of overload resolution to determine whether a default comparison should be generated.

Examples

In the examples, cases where currently well-formed code becomes ill-formed or changes semantics is highlighted in **red**.

Basic case

```
struct B { int x; };
B b;
bool result = B() == b;   // ok, default== for B
```

No slicing with defaults

```
struct B { int x; };
struct D : B { int y; };
B b1, b2;
D d1, d2;

B b3(b1);             // ok, direct-initialization
B b4(d1);             // error, violates R1 for the implicit copy constructor of B
B b5(static_cast<B&>(d1));   // ok, binds parameter of copy constructor to "lvalue of type B"
D d3(d1);             // ok, direct-initialization
D d4(b1);             // error, can't convert B to D

int f(B);
int i1 = f(b1);       // ok, copy-initialization
int i2 = f(d1);       // error, violates R1 for the implicit copy
int i3 = f(static_cast<B&>(d1));  // ok, binds parameter of copy constructor to "lvalue of type B"

int g(const B&);
int i4 = g(b1),       // ok, direct reference binding
int i5 = g(d1);       // ok, direct reference binding to B subobject of d1

void h() {
  b2 = b1;            // ok, copy-assignment
  b2 = d1;            // error, violates R1 for the implicit copy-assignment of B
  b2 = static_cast<B&>(d1);   // ok, binds parameter of copy-assignment operator to "lvalue of type B"
  d2 = d1;            // ok, copy-assignment
  d2 = b1;            // error, can't convert B to D
}

bool r1 = b1 == b2;   // ok, default== for B
bool r2 = d1 == d2;   // ok, default== for D
bool r3 = b1 == d2;   // error: violates R1 for the default== for B
```

No slicing with user-declared functions

```
struct B {
  B(const B&);                // #1
  B& operator=(const B&);     // #2
  int x;
};
bool operator==(const B&, const B&);
struct D : B { int y; };
B b1, b2;
D d1, d2;

B b3(b1);              // ok, direct-initialization
B b4(d1);              // error, violates R1 for the call to #1
D d3(d1);              // ok, direct-initialization, calls #1 for copying the B subobject of d1
D d4(b1);              // error, can't convert B to D

int f(B);
int i1 = f(b1);        // ok, copy-initialization
int i2 = f(d1);        // error, violates R1 for the implicit copy

int g(const B&);
int i3 = g(b1),        // ok, direct reference binding
int i4 = g(d1);        // ok, direct reference binding to B subobject of d1

void h() {
  b2 = b1;            // ok, copy-assignment
  b2 = d1;            // error, violates R1 for the call to #2
  d2 = d1;            // ok, copy-assignment
  d2 = b1;            // error, can't convert B to D
}

bool r1 = b1 == b2;    // ok, operator== for B
bool r2 = d1 == d2;    // ok, default== for D, invokes operator== for B subobjects
bool r3 = b1 == d2;    // error: violates R1 for user-declared operator==
```

Benign slicing

This continues to allow tag dispatch for differentiating iterator categories, for example.

```
struct B { int x; }
struct D : B { };   // no additional subobjects vs. B

B b1, b2;
D d1, d2;

B b3(b1);             // ok, direct-initialization
B b4(d1);             // ok, slicing is harmless and permitted
D d3(d1);             // ok, direct-initialization
D d4(b1);             // error, can't convert B to D

int f(B);
int i1 = f(b1);       // ok, copy-initialization
int i2 = f(d1);       // ok, slicing is harmless and permitted

int g(const B&);
int i4 = g(b1),       // ok, direct reference binding
int i5 = g(d1);       // ok, direct reference binding to B subobject of d1

void h() {
  b2 = b1;            // ok, copy-assignment
  b2 = d1;            // ok, slicing is harmless and permitted
  d2 = d1;            // ok, copy-assignment
  d2 = b1;            // error, can't convert B to D
}

bool r1 = b1 == b2;   // ok, default== for B
bool r2 = d1 == d2;   // ok, default== for D
bool r3 = b1 == d2;   // ok, default== for B (see R2; slicing allowed by R1)
```

Hiding by user-declared functions I

```
struct S {
  bool operator==(const S&);   // #1; note: non-const (user oversight)
};

S s;
bool b1 = s == s;               // calls #1
bool b2 = S() == S();           // error, can't call #1 and no default==

struct S2 { };
bool operator==(const S2&, const S2&);  // #2

bool b3 = S2() == S2();         // calls #2

struct S3 { };
S operator==(const S3&, S3&);   // #3 (strange)
S3 s3;
S b4 = s3 == s3;                // calls #3; no default==
S b5 = S3() == S3();            // error, can't call #3 and no default==
```

Hiding by user-declared functions II

```
struct S { };
namespace N {
  bool operator==(const S&, const S&);  // #1
  bool operator==(const S&, S&);        // #2
  bool b = S() == S();   // calls #1 (note: some prefer default== or error)
  S s;
  bool b2 = S() == s;    // calls #2 (note: some prefer default== or error)
}
```

Hijacking declarations I

```
struct S { };
bool b1 = S() == S();     // ok, use default==
namespace N {
  bool operator==(S,S);   // #1
  bool b2 = S() == S();   // ok, use N::operator==
}
```

Hijacking declarations II

```
struct S { };              // #1
struct S2 { S s; };
```

```
namespace N {
  bool operator==(S,S);    // #2
  bool b = S2() == S2();   // #3  error
}
```

For #3, we use the default== on S2. Its definition uses the default== on S introduced at #1, but that violates R5 at #2.

User-declared conversions

```
struct S {
  operator int() const;
};
bool b = S() == S();        // ok, default== for S
```

Templates I

```
struct C { };
template<class T>
bool operator==(const T&, const T&);
bool b = C() == C();           // ok, use operator== template
```

Templates II

```
template<class T>
class S { };
template<class T>
bool operator==(const S<T>&, const S<T>&);
S<int> s;
bool b = s == s;       // use user-declared operator==
```

```
template<class T>
class S2 {
  friend bool operator==(const S2<T>&, const S2<T>&) { ... }
};
S<int> s;
bool b2 = s == s;      // uses user-declared operator==
```

Templates III (examples by Herb Sutter)

Case 1: User-written == appears as a member of C, or in the same namespace as C and mentions C specifically. All of these cases call the user-written ==. Varying the parameter types between C, const C&, and C& does not change the outcome, except that the rules preventing binding prvalues to lvalue references remain in force.

```
// case 1a: obvious case
struct C1 {
  bool operator==(const C1&) const;
};
bool b = C1() == C1();                // uses user-declared == for C1

// case 1b: obvious case, non-member
struct C2 { };
bool operator==(const C2&, const C2&);
bool b = C2() == C2();                // uses user-declared == for C2

// case 1c: class template member
template<class T>
struct C3 {
  bool operator==(const C3&) const;
};
bool b = C3() == C3();                // uses user-declared == for C3

// case 1d: function template for a class template
template<class T> struct C4 { };
template<class T> bool operator==(const C4<T>&, const C4<T>&);
bool b = C4() == C4();                // uses user-declared == for C4


// case 2a: fully general template
struct C5 { };
template<class T> bool operator==(const T&, const T&);
C5 c5;
bool b = C5() == C5();              // calls operator function template

// case 2b: same with concept
struct C6 { };
template<My_concept T> bool operator==(const T&, const T&);
```

```
bool b = C6() == C6();              // calls operator function template

// case 2c: conversion
struct C7 { };
struct X { X(const C7&){} };         // convertible from C7
bool operator==(const X&, const X&);
bool b = C7() == C7();               // uses default== for C7
```

Open Issues

(none at this time)

Closed Issues

- We generate != from == even if the class doesn't satisfy the constraints. This would only be possible if a user-defined operator== exists, of course. We generate any relational operators other than < if both user-defined == and < exist. Example:

```
struct S {
  int i = 0;
};
bool operator==(const S&, const S&)
{ return true; }

int f() {
  S() != S();    // well-formed, yields false
}
```

- A class with a mutable member does not get a memberwise operator== or operator< (deviating from N4475 page 17).
- `struct S { int a[3]; }` will acquire generated comparison functions, but top-level (standalone) `int[3]` (i.e. an array not wrapped in a struct) will not.
- "T has operator op declared" is interpreted to mean: there is a viable function for x op y, where x and y are lvalues of type T. Examples:

```
struct A { };
void operator==(const A&, const A&);            // yes
void operator==(const A&, int);                 // no
void operator==(A&, const A&);                  // yes
void operator==(A&&, volatile A&);              // no
void operator==(const volatile A&&, const A&);  // no
void operator==(A, const A&);                   // yes
template<class T>
void operator==(const T&, A);                   // yes
template<class T>
void operator==(const T&, const T&);            // yes
template<class T>
void operator==(T&&, T&&);                      // no
```

- What kind of lookup is performed for the operator function name when determining that "a class has a given operator declared"? Usual unqualified lookup from which context? Just argument-dependent lookup? Something else? Example:

```
struct A { int x, y; } a;
namespace N {
  void *operator<(A, A);
  auto q = a > a;
}
```

  Consistent with aim 2, we cannot use N::operator<. This is ill-formed to avoid silent surprises.
- Generation of memberwise operator< relies on memberwise operator==. We do not generate operator< for a class C (implicitly using memberwise == in its definition) if we have a user-declared operator== for C. The latter expresses special comparison semantics for class C, so it's likely that operator< needs to be special as well. Example:

```
struct S {
  int x;
};
bool operator==(const S&, const S&);

S s;
bool b = s < s;    // error
```

- Deviating from the recommendation in N4367 "Comparison in C++" by Lawrence Crowl, there is no special case for classes with a single member.

Wording

Add at the end of 3.9.2 [basic.compound]:

> **A type *cv* T *supports comparison by subobject* if T is an array type, or is a non-union class type that satisfies the following constraints:**
>
> - **T has no virtual base class (clause 10 [class.derived]),**
> - **T has no virtual member function (10.3 [class.virtual]),**
> - **T has no direct mutable member (7.1.1 [dcl.stc]),**
> - **T has no direct non-static data member of pointer type (9.2 [class.mem]), and**
> - **T has no user-provided or deleted copy/move constructor or copy/move assignment operator (12.8 [class.copy]).**
>
> **T *supports operator op* in a given context if overload resolution (13.3.1.2 [over.match.oper]) finds a viable function for the expression `x` *`op`* `y`, where x and y are lvalues of type T.**
>
> **T *supports equality comparison by subobject* in a given context if T supports comparison by subobject and does not support operator== and operator!= in that context.**
>
> **T *supports relational comparison by subobject* in a given context if T supports equality comparison by subobject in that context and does not support operator <, operator <=, operator >, and operator >= in that context.**

Change in 9 [class] paragraph 7 bullet 6:

- ...
- has all non-static data members and bit-fields in the class and its base classes ~~first declared in~~ **as direct members of** the same class, and
- ...

Change in 9.2 [class.mem] paragraph 1:

> The *member-specification* in a class definition declares the full set of members of the class; no member can be added elsewhere. **A *direct member* of a class is a member of that class that was first declared within the class's *member-specification*.** ...

Add a new section 9.10 [class.oper]:

> **9.10 Operators [class.oper]**
>
> **[Note: This section specifies the meaning of relational (5.9 [expr.rel]) or equality (5.10 [expr.eq]) operators applied to values of class or array type.]**
>
> **This section defines the result of comparing two objects x and y that have the same class or array type T (ignoring cv-qualification) by decomposing x and y into a sequence of corresponding subobjects and comparing the pairs of corresponding subobjects. The context for applying a comparison operator to such a pair is defined as follows: If the original x-y pair is compared, the context is where the comparison appears. Otherwise, the context is as if in the body of a friend function defined in the definition of T. If the corresponding subobjects have a type that was instantiated from a dependent type, the operator is considered to have been applied to type-dependent expressions. [Note: The context determines operator function lookup and access control. The provision for templates ensures that the usual two-phase name lookup rules apply (14.6 [temp.res]).]**
>
> **When comparing the original x-y pair of corresponding subobjects using == or <, if a viable function is not a member of T or a member of the innermost enclosing namespace of T, the program is ill-formed.**
>
> **Comparing x and y for equality is defined as follows:**
>
> - **A sequence of corresponding subobjects of x and y is formed by starting with a sequence comprising x corresponding to y, and repeatedly replacing each element whose type supports equality comparison by subobject (3.9.2 [basic.compound]) with a sequence of the corresponding subobjects of the direct base classes and direct members of that type, in order of declaration or order of increasing array subscript.**
> - **If the resulting sequence of corresponding subobjects has no elements, `x == y` yields true and `x != y` yields false.**
> - **Otherwise, for each element in the sequence of corresponding subobjects, the corresponding subobjects are compared using ==. If this comparison, when contextually converted to `bool`, yields false, no further**

subobjects are compared, and `x == y` yields false and `x != y` yields true. If all such comparisons yield true, then `x == y` yields true and `x != y` yields false.

Comparing x and y with a relational operator is defined as follows:

- **For x > y, the result is y < x. For x >= y, the result is y <= x.**
- **A sequence of corresponding subobjects of x and y is formed by starting with a sequence comprising x corresponding to y, and repeatedly replacing each element whose type supports relational comparison by subobject (3.9.2 [basic.compound]) with a sequence of the corresponding subobjects of the direct base classes and direct members of that type, in order of declaration or order of increasing array subscript.**
- **If the resulting sequence of corresponding subobjects has no elements, `x < y` yields false and `x <= y` yields true.**
- **Otherwise, for each element in the sequence of corresponding subobjects, the corresponding subobjects are compared using ==. If this comparison, when contextually converted to bool, yields false for the first time, that element is the first differing one. If the final element is reached for a < comparison, that element is the first differing one, and is not compared with ==. Otherwise, if all such comparisons yield true, there is no first differing element.**
- **If there is no first differing element, the result of the <= comparison is true. Otherwise, the corresponding subobjects of the first differing element are compared using <. The result of this comparison, when contextually converted to bool, is the result of the overall comparison.**

[Example:

```
struct B {
  int i = 0;
};

struct S : B {
  int j = 1;
};

struct T : S {
   bool operator>(T) = delete;
};

void f()
{
  B b;
  S s1, s2;
  s2.j = 2;
  s1 == s1;      // yields true
  s1 != s2;      // yields true
  s1 < s1;       // yields false
  s1 < s2;       // yields true
  s1 == b;       // error: not the same class type
  T() == T();    // yields true
  T() < T();     // error: operator> is user-declared
}

template<class T>
struct V {
  T x;
};

struct E { };
bool operator==(E,E);

V<E> v;
bool b = v == v;   // ok, default== finds operator== for E
```

-- end example]

Add a paragraph before 13.3.1.2 [over.match.oper] paragraph 5:

For a relational (5.9 [expr.rel]) or equality (5.10 [expr.eq]) operator where both operands are of the same class type, no user-defined conversions are applied to any of the operands.

For all other operators, no such restrictions apply.

Add a paragraph after 13.3.1.2 [over.match.oper] paragraph 7:

If overload resolution yields no viable function (13.3.2 [over.match.viable]) for a relational (5.9 [expr.rel]) or

equality (5.10 [expr.eq]) operator and the operands are of the same complete class type T (ignoring cv-qualification), then:

- **If the operator is == and T does not support equality comparison by subobject, the program is ill-formed.**
- **If the operator is < and T does not support relational comparison by subobject, the program is ill-formed.**
- **Otherwise, the operator is treated as a built-in operator and interpreted according to 9.10 [class.oper].**

If a comparison operator is treated as a built-in operator in a context whose nearest enclosing namespace is N, and a comparison with the same operator and the same operand types in another context whose nearest enclosing namespace is also N is not treated as a built-in operator, the program is ill-formed. No diagnostic is required if the two comparisons appear in different translation units. [Example:

```
struct S { };
bool b = S() == S();   // built-in ==

bool operator==(const S&, const S&);
S s;
bool b2 = s == s;      // error: not using built-in ==
```

-- end example]

Change in 13.3.2 [over.match.viable] paragraph 3:

Second, for F to be a viable function, there shall exist for each argument an implicit conversion sequence (13.3.3.1) that converts that argument to the corresponding parameter of F. If the parameter has reference type, the implicit conversion sequence includes the operation of binding the reference, and the fact that an lvalue reference to non-const cannot be bound to an rvalue and that an rvalue reference cannot be bound to an lvalue can affect the viability of the function (see 13.3.3.1.4). **If F is a**

- **copy/move constructor (12.8 [class.copy]),**
- **copy/move assignment operator function, or**
- **relational (5.9 [expr.rel]) or equality (5.10 [expr.eq]) operator function,**

an implicit conversion sequence shall not bind a parameter of type "reference to *cv* B" to an argument of class type D derived from B, unless all non-static data members of D are inherited from B (if any) and D has no virtual functions and no virtual base classes.

Add a new section to Annex C:

C.4.4 Clause 13: Overloading [diff.cpp14.over]

13.3.1.2 [over.match.oper]
Change: Disallow derived-to-base and user-defined conversions for comparison operators
Rationale: Avoid a common source of errors.
Effect on original feature: Valid C++ 2014 code may fail to compile or change meaning in this International Standard:

```
struct B { };
bool operator==(const B&, const B&);
struct D : B { int *p; };
bool b1 = D() == B();     // ill-formed; previously well-formed
bool b2 = D() == D();     // ill-formed; previously well-formed
struct S { operator int() const; int * p; };
bool b3 = S() == S();     // ill-formed; previously well-formed
```

13.3.2 [over.match.viable]
Change: Disallow derived-to-base conversions for copy construction and copy assignment ("slicing").
Rationale: Avoid a common source of errors.
Effect on original feature: Valid C++ 2014 code may fail to compile or change meaning in this International Standard:

```
struct B { };
struct D : B { int x = 0; };
D d;
B b1 = d;     // ill-formed; previously well-formed
B b2 = static_cast<const B&>(d);   // well-formed
```

Acknowledgements

Thanks to Richard Smith for valuable input, and the majority of the drafting presented above. All errors are mine, of course.

Thanks to Bjarne Stroupstrup and Herb Sutter for in-depth discussions.